

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria

Dipartimento di Ingegneria Civile e Ingegneria Informatica (DICII)

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

INGEGNERIA DEL SOFTWARE E PROGETTAZIONE WEB (ISPW)

CIBOAMICO

*PIATTAFORMA PER LA GESTIONE DEL CIBO IN CASA, LA RIDUZIONE
DEGLI SPRECHI E L'OTTIMIZZAZIONE DELLA SPESA*

DOCENTI:

PROF. DAVIDE FALESSI

PROF. GUGLIELMO DE ANGELIS

AUTORE:

MICHELE DAMIANO

MATRICOLA: 0324516

MICHELE.DAMIANO@STUDENTS.UNIROMA2.EU

ANNO ACCADEMICO 2025/2026

DATA DI CONSEGNA: 8 AGOSTO 2026

Sommario

CiboAmico risolve il problema dello spreco alimentare domestico e della frammentazione del commercio a chilometro zero. La piattaforma offre una gestione intelligente della dispensa, un motore di matching delle ricette con sostituzione automatica degli ingredienti e un marketplace locale per l'interazione diretta tra clienti, commercianti e amministratori. L'architettura software si basa sul pattern *Boundary-Control-Entity* (BCE). Ho implementato una doppia interfaccia utente intercambiabile a runtime—un'interfaccia grafica in JavaFX e una riga di comando (CLI)—orchestrata tramite una **ViewFactory** e disaccoppiata dai controller applicativi stateless mediante l'uso di Bean (DTO).

Per la persistenza ho progettato tre modalità (JDBC MySQL, File System JSON e Demo in-memory) gestite da un'Abstract Factory di DAO. Ho inoltre applicato diversi pattern GoF, tra cui *Strategy* per la logica di matching, *Facade*, *Observer* per la gestione degli ordini, *Singleton* e due *Adapter* per OpenFoodFacts e JakartaMail, garantendo la sicurezza delle credenziali tramite hashing SHA-256 e salt. La correttezza e la qualità della logica di business sono verificate da 92 unit test JUnit 5, con una Line Coverage dell'81.4%, e dall'assenza di violazioni rilevate su SonarCloud.

Indice

Sommario	1
1 Specifiche dei Requisiti Software	5
1.1 Introduzione	5
1.1.1 Scopo del documento	5
1.1.2 Panoramica del sistema	5
1.1.3 Requisiti hardware e software	6
1.1.4 Sistemi correlati, pro e contro	6
1.2 User Stories	7
1.3 Requisiti Funzionali	7
1.3.1 Glossario dei Termini di Dominio	7
1.4 Use Cases	8
1.4.1 Diagramma dei casi d'uso	8
1.4.2 Internal Steps	8
2 Storyboards	10
2.1 Autenticazione e Area Personale	10
2.1.1 Autenticati	10
2.1.2 Home (Cliente)	10
2.2 Acquisti e Marketplace (UC-04 Ordina un Prodotto)	11
2.2.1 Marketplace (Ricerca Prodotti)	11
2.2.2 Ordina un Prodotto	12
2.2.3 Ordina un Prodotto con Buono Promozionale	12
2.2.4 Pagamento (passo 6)	13
3 Design	14
3.1 View of Participating Classes (VOPC)	14
3.2 Design-Level Class Diagram	16
3.2.1 Abstract Factory: persistenza (NFR-01)	16
3.2.2 Pattern Strategy: scontistica del buono	16
3.2.3 Pattern Observer: notifica al venditore	17

3.2.4	Pattern Singleton: sessione e modalità	18
3.2.5	Macchina a stati dell'Ordine (BR-04)	18
3.3	Modellazione Comportamentale	18
3.3.1	Activity Diagram (Ordina un Prodotto)	18
3.3.2	Sequence Diagram (Ordina un Prodotto)	20
3.3.3	State Diagram (Ordina un Prodotto, navigazione UI)	21
3.4	Realizzazione del Buono Promozionale	23
4	Testing	24
4.1	Strumenti e organizzazione	24
4.2	Specifiche dei casi di test	24
4.2.1	Bean e DTO	24
4.2.2	Controller applicativi	25
4.2.3	Entità e dominio	25
4.2.4	Pattern e factory	25
4.2.5	Persistenza	26
4.3	Tracciabilità Requisiti-Test	26
4.4	Risultati dell'esecuzione	26
4.5	Copertura per sotto-sistema	27
4.6	SonarCloud e quality gate	27
5	Exceptions	28
5.1	BusinessValidationException	28
5.2	AutenticazioneException	28
5.3	InvalidStateTransitionException	29
5.4	Gestione nelle Boundary	29
6	Persistenza	30
6.1	Pattern Abstract Factory	30
6.2	Modalità Demo (in-memory)	30
6.3	Modalità Filesystem (JSON)	31
6.4	Modalità JDBC (MySQL)	31
6.5	Coerenza con il sequence e gli use case	31
7	Codice e Qualità	32
7.1	Struttura del codice	32
7.2	Metodologia di sviluppo	33
7.3	Verifica dei design pattern GoF	33
7.4	Verifica e quality gate	34
7.5	Correzioni architetturali applicate	34

8	Video e Link	35
8.1	Repository GitHub	35
8.2	SonarCloud	35
8.3	Video dimostrativo	35

Capitolo 1

Specifiche dei Requisiti Software

1.1 Introduzione

1.1.1 Scopo del documento

Questo documento definisce i requisiti software di **CiboAmico**, il progetto finale del corso ISPW (Università di Roma “Tor Vergata”, A.A. 2025/2026, Prof. Davide Falessi, Prof. Guglielmo De Angelis).

Lo scopo è fissare cosa il sistema deve fare in modo chiaro e verificabile, lasciando campo alla progettazione e ai test. Il documento serve sia a chi sviluppa, per tenere fede all’architettura Boundary-Control-Entity (BCE), sia a chi valuta il prodotto, per confrontare i flussi funzionali con le richieste iniziali.

1.1.2 Panoramica del sistema

CiboAmico è un’applicazione desktop JavaFX che aiuta a gestire il cibo in casa, evitare sprechi e ottimizzare la spesa, collegando l’utente ai commercianti di prossimità. Il sistema segue il pattern Boundary-Control-Entity (BCE) e coinvolge quattro attori: Cliente, Venditore, Nutrizionista e Amministratore. Due sistemi esterni sono raggiunti tramite Adapter: OpenFoodFacts (lettura dei dati nutrizionali dal codice a barre) e JakartaMail (invio di notifiche). La persistenza è intercambiabile a runtime tra una modalità *Demo* (in-memory) e una *Full* (MySQL via JDBC).

Il sistema ha tre aree funzionali:

- **Dispensa:** il Cliente traccia i prodotti in casa, vede le scadenze e viene avvisato prima che il cibo si deteriori;
- **Ricette:** il sistema propone ricette compatibili con gli ingredienti disponibili, sostituendo quelli mancanti;

- **Marketplace:** i Venditori pubblicano i prodotti, i Clienti fanno ordini diretti (un compratore, un venditore) e l'Amministratore approva venditori e ricette.

1.1.3 Requisiti hardware e software

L'applicazione è cross-platform grazie alla JVM. I requisiti minimi sono: processore dual-core a 64-bit, 2 GB di RAM, 150 MB di disco e schermo 1024×768 . In modalità JDBC, che esegue anche il server MySQL locale, si consigliano 4 GB di RAM, 500 MB su SSD e una scheda grafica compatibile con OpenGL 2.0.

A livello software servono: OpenJDK 17 LTS, JavaFX 17+, Maven 3.9+ e, per la modalità database, MySQL 8.0+. Per sviluppo e controllo qualità si usano IntelliJ IDEA (con EclEmma per la coverage), JUnit 5, SonarCloud e GitHub Actions. I diagrammi UML sono realizzati con StarUML o Draw.io.

1.1.4 Sistemi correlati, pro e contro

Di seguito due sistemi esistenti che coprono solo in parte quello che fa CiboAmico.

Too Good To Go Piattaforma B2C per il recupero delle eccedenze alimentari: connette l'utente con commercianti e supermercati locali.

- **Pro:** rete capillare di partner con offerte geolocalizzate in tempo reale; riduce attivamente lo spreco commerciale offrendo cibo a prezzi ridotti.
- **Contro:** non traccia la dispensa di casa, quindi dopo l'acquisto non aiuta a gestire le scadenze; l'acquisto è una *Magic Box* a sorpresa, non si scelgono i singoli prodotti né si pianificano ricette o diete.

SuperCook Motore di ricerca di ricette in base agli ingredienti già in dispensa.

- **Pro:** matching potente che trova subito molte ricette a partire dagli ingredienti inseriti; ampio database con filtri per tipo di piatto e intolleranze.
- **Contro:** non ha un canale di acquisto integrato, quindi per gli ingredienti mancanti bisogna provvedere da soli; non sostituisce automaticamente gli ingredienti, escludendo ricette realizzabili con piccole variazioni.

Posizionamento di CiboAmico CiboAmico unisce i due approcci: gestisce la dispensa contro lo spreco (come SuperCook), propone ricette con sostituzioni intelligenti e, se manca qualcosa, permette di comprarlo dai produttori locali a chilometro zero (come Too Good To Go), tutto in una sola applicazione.

1.2 User Stories

1. **US-1:** Come Cliente, voglio ricevere notifiche preventive sulle date di scadenza dei prodotti presenti nella mia dispensa, in modo che possa consumare gli alimenti in tempo ed evitare lo spreco di cibo.
2. **US-2:** Come Cliente, voglio visualizzare le ricette compatibili con gli ingredienti disponibili in dispensa, in modo che possa cucinare senza dover effettuare nuovi acquisti.
3. **US-3:** Come Cliente, voglio ordinare un prodotto direttamente da un venditore locale, in modo che possa ricevere cibo fresco proveniente dal mio territorio.

1.3 Requisiti Funzionali

1. **RF-1:** Il sistema deve confrontare quotidianamente le date di scadenza degli alimenti nella dispensa con la data corrente, inviando un avviso al cliente per ogni prodotto la cui scadenza sia entro tre giorni.
2. **RF-2:** Il sistema deve suggerire al cliente un elenco di ricette in cui gli ingredienti richiesti siano un sottoinsieme degli ingredienti non scaduti presenti nella dispensa.
3. **RF-3:** Il sistema deve registrare una nuova transazione di acquisto associando il cliente al venditore e decrementando la scorta del prodotto in misura pari alla quantità ordinata.

1.3.1 Glossario dei Termini di Dominio

Dispensa : rappresentazione digitale degli alimenti presenti nell'inventario domestico dell'utente, con quantità e date di scadenza.

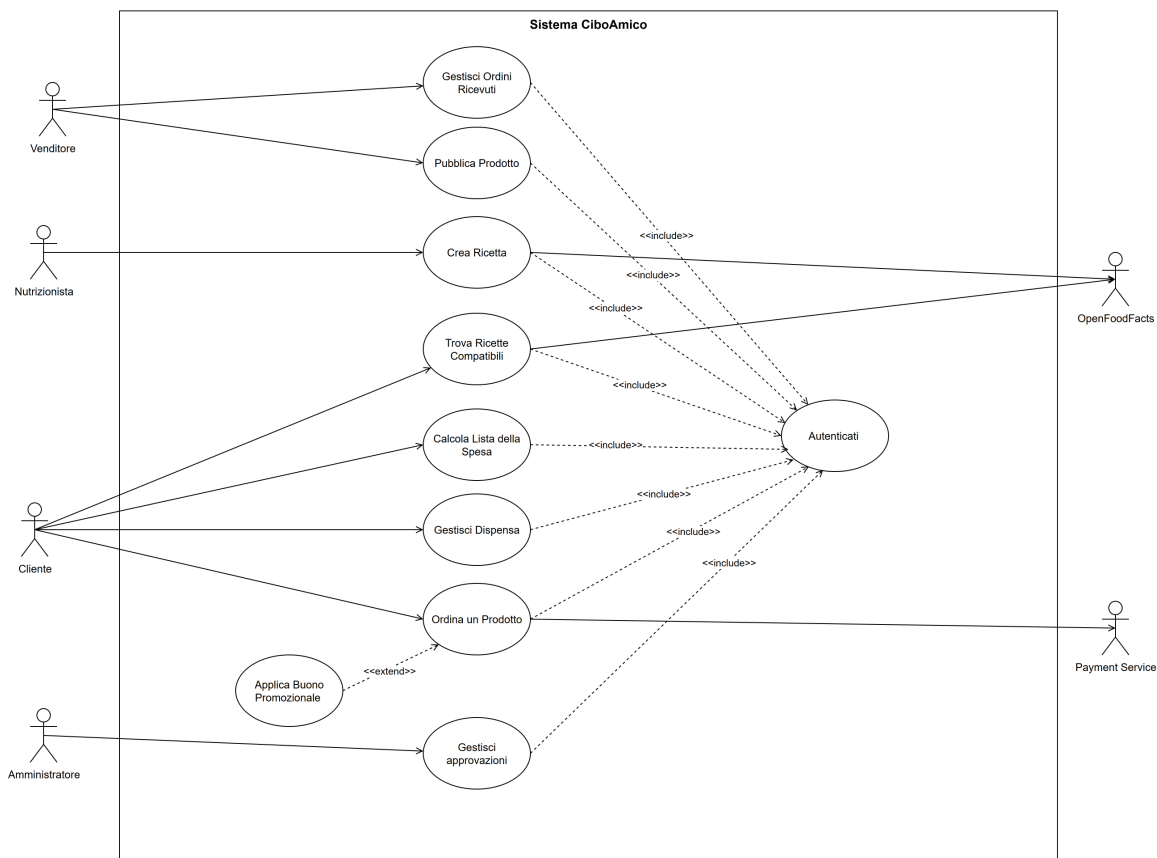
Ricetta compatibile : ricetta i cui ingredienti richiesti sono presenti, oppure sostituibili in modo equivalente, nella dispensa dell'utente.

Ordine diretto : transazione che associa un singolo cliente acquirente a un unico venditore, senza passare da un carrello condiviso o intermediari.

Disponibilità del prodotto : quantità di un prodotto attualmente vendibile, che non può mai essere negativa e viene aggiornata dopo ogni acquisto.

1.4 Use Cases

1.4.1 Diagramma dei casi d'uso



Il caso d'uso **Ordina un Prodotto** è stato progettato con un grado di astrazione che consente di modellare funzionalità distinte. In questa implementazione, il caso d'uso secondario **Applica Buono Promozionale** è associato al caso d'uso primario tramite una relazione di *extend*: l'attore **Cliente** può utilizzare questa funzionalità estesa solo se si autentica correttamente nel sistema (relazione di *include* verso **Autenticati**) e proseguendo poi il flusso del caso d'uso **Ordina un Prodotto**. Analogamente, **Autenticati** è incluso da più casi d'uso, a sottolinearne il riuso nell'ambito delle funzionalità principali.

1.4.2 Internal Steps

Ordina un Prodotto

Scenario principale:

1. Il Sistema individua i prodotti disponibili sul marketplace con prezzi, scorte residue e venditori.
2. Il Cliente richiede di aggiungere un prodotto all'ordine, indicandone la quantità desiderata.

3. Il Sistema convalida che la quantità richiesta non superi la scorta residua del prodotto.
4. Il Sistema calcola l'importo totale della transazione.
5. Il Cliente richiede di confermare l'acquisto.
6. Il Sistema richiede al Payment Service l'autorizzazione all'addebito per l'importo calcolato.
7. Il Sistema registra l'ordine in attesa di approvazione e decrementa la scorta residua dei prodotti ordinati.
8. Il Sistema invia una notifica al Venditore per segnalare l'ordine ricevuto.

Estensioni:

- **3a. Superamento della scorta residua:**

1. Il Sistema rileva che la quantità inserita dal Cliente è superiore alla disponibilità corrente del prodotto.
2. Il Sistema segnala l'anomalia al Cliente indicando la massima quantità acquistabile.
3. Il flusso di controllo ritorna al passo 2.

- **4a. Richiesta di applicazione di un buono promozionale (tramite l'estensione del caso d'uso Applica Buono Promozionale):**

1. Il Cliente richiede di applicare un codice di buono promozionale all'ordine corrente.
2. Il Sistema convalida la validità temporale del buono promozionale e la sua associazione con il Venditore del prodotto selezionato.
3. Il Sistema ricalcola l'importo totale della transazione applicando lo sconto previsto dal buono promozionale.
4. Il flusso di controllo prosegue dal passo 5 dello scenario principale.

- **6a. Autorizzazione di pagamento negata:**

1. Il Sistema riceve l'esito negativo dall'autorizzazione del Payment Service.
2. Il Sistema segnala al Cliente il fallimento del pagamento.
3. Il flusso di controllo ritorna al passo 5.

Capitolo 2

Storyboards

2.1 Autenticazione e Area Personale

2.1.1 Autenticati

La schermata di accesso permette all'utente di autenticarsi inserendo le proprie credenziali (email e password).

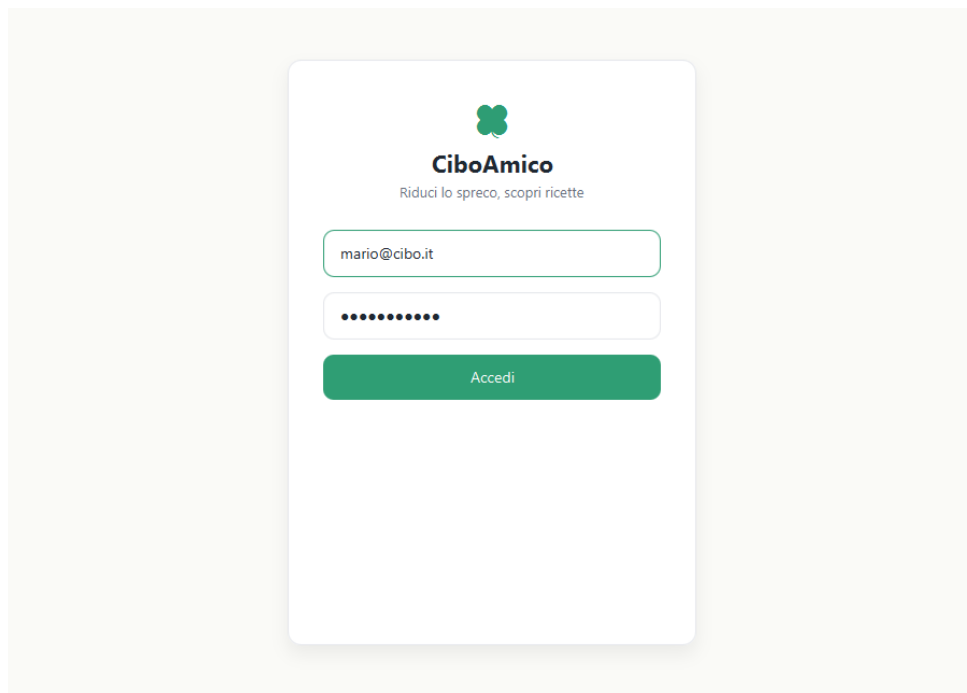


Figura 2.1: Schermata di autenticazione al sistema.

2.1.2 Home (Cliente)

Completata l'autenticazione, il Cliente raggiunge la dashboard principale; nell'ambito dello scope UC-04 la Home offre l'accesso diretto al Marketplace per ordinare un prodotto locale.

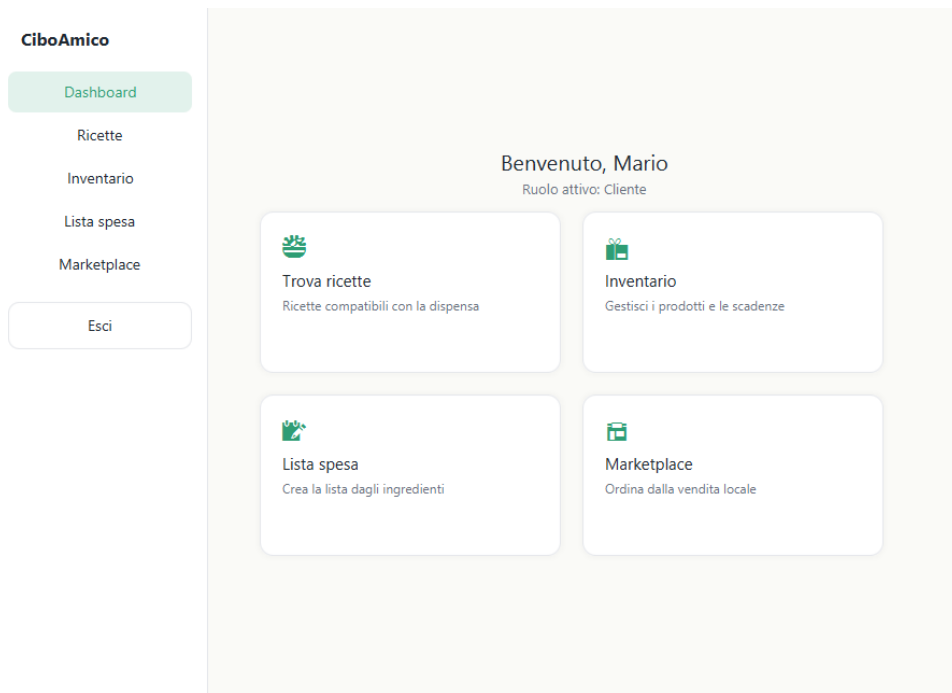


Figura 2.2: Dashboard principale (accesso al Marketplace).

2.2 Acquisti e Marketplace (UC-04 Ordina un Prodotto)

2.2.1 Marketplace (Ricerca Prodotti)

L'area dedicata all'acquisto diretto mostra i prodotti locali dei venditori, con prezzo e disponibilità residua.

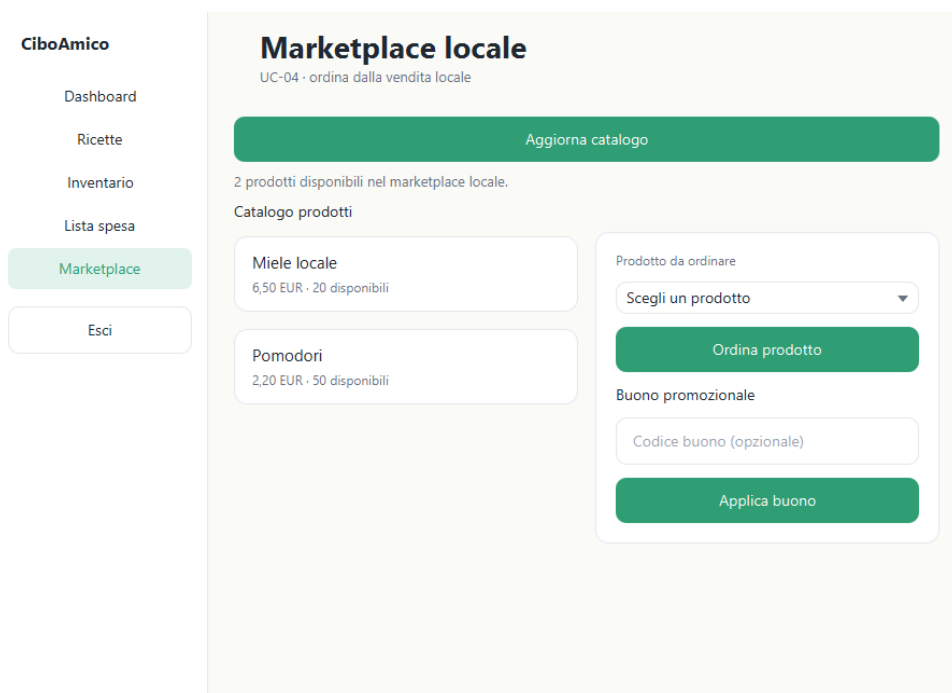


Figura 2.3: Catalogo dei prodotti locali disponibili per l'acquisto.

2.2.2 Ordina un Prodotto

Il sistema presenta il riepilogo dell'ordine selezionato e consente di procedere con l'acquisto.

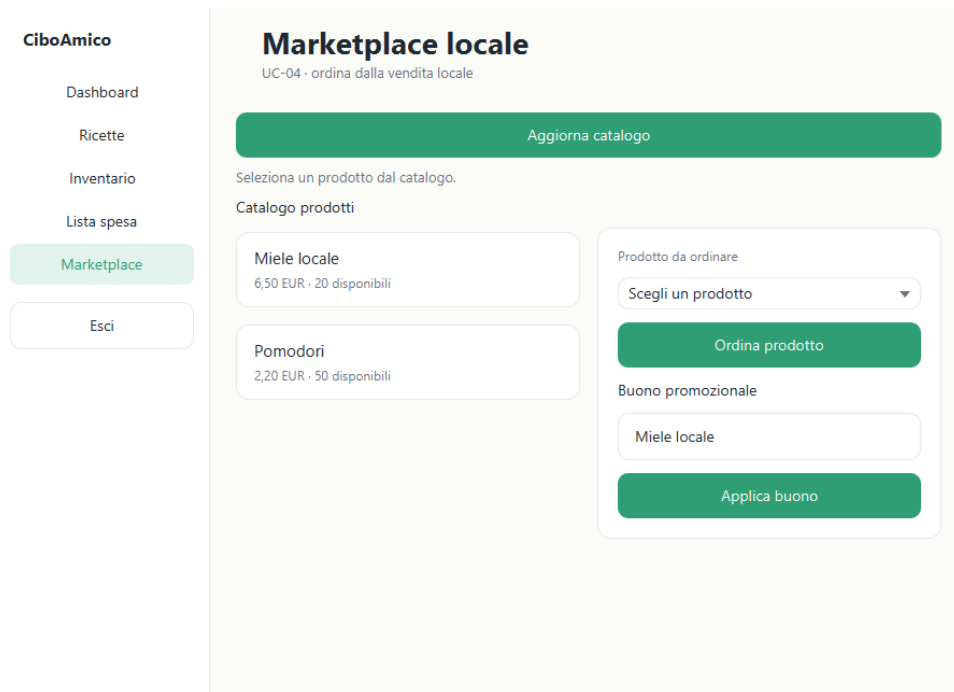


Figura 2.4: Schermata dell'ordine e del riepilogo.

2.2.3 Ordina un Prodotto con Buono Promozionale

L'interfaccia consente di inserire un codice di buono promozionale: il sistema valida il buono e ricalcola il totale applicando lo sconto (estensione del caso d'uso *Ordina un Prodotto*).

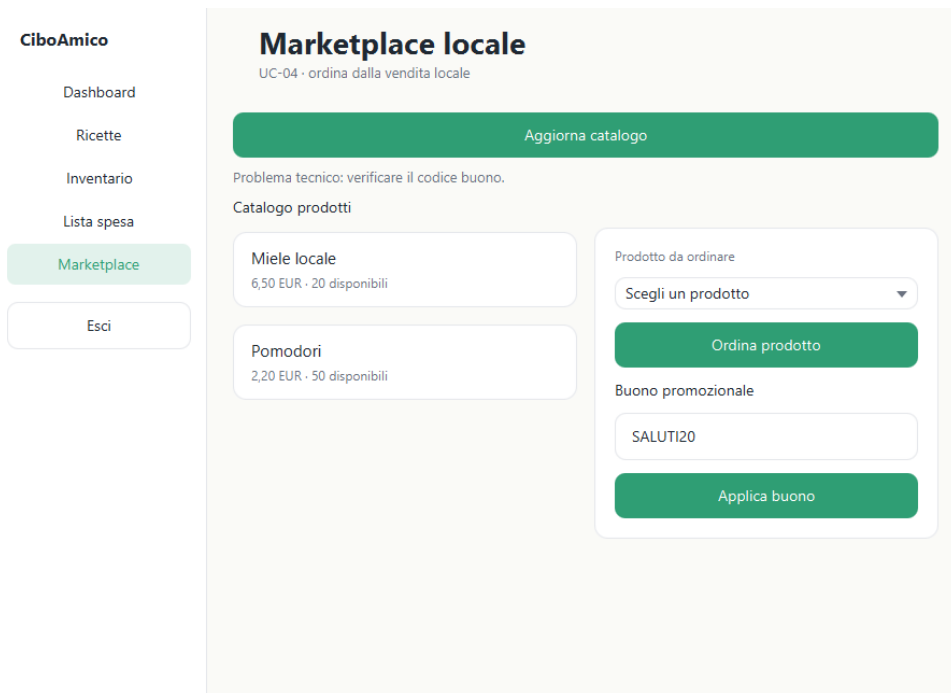


Figura 2.5: Applicazione di un buono promozionale e totale scontato.

2.2.4 Pagamento (passo 6)

Confermato l'ordine, l'interfaccia passa alla schermata di pagamento: l'utente inserisce i dati della carta (numero, intestatario, scadenza e CVV) e autorizza l'addebito; l'esito è mostrato inline nella stessa vista, come richiesto dall'estensione **6a** (pagamento rifiutato) del caso d'uso.

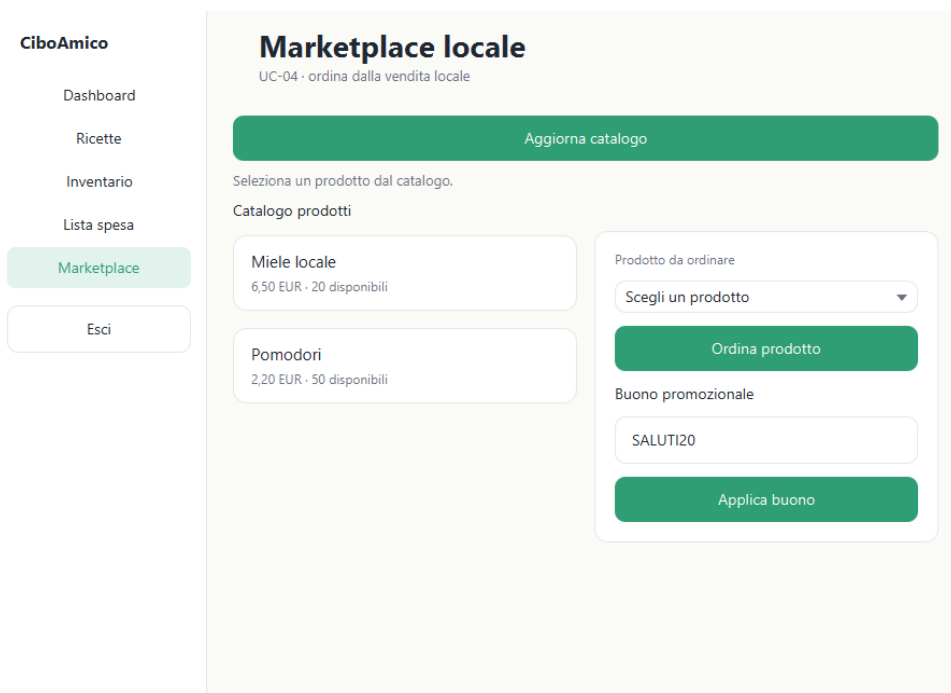


Figura 2.6: Schermata di pagamento (dati carta e autorizzazione all'addebito).

Capitolo 3

Design

Il presente capitolo illustra la progettazione di **CiboAmico** per il caso d'uso **Ordina un Prodotto** (UC-04): la vista di analisi (VOPC), il diagramma delle classi di progettazione con i pattern GoF applicati e la modellazione comportamentale (activity, sequence e state) dell'UC-04.

3.1 View of Participating Classes (VOPC)

Il **VOPC** è il diagramma delle classi di analisi che mostra *solo* i partecipanti del caso d'uso **Ordina un Prodotto** (UC-04). L'architettura segue il pattern **Boundary–Control–Entity** puro (pre-MVC):

- ogni caso d'uso possiede **esattamente un** controller applicativo;
- la boundary media l'interazione con l'attore senza esporre dettagli della GUI (Lezione 24);
- il controller è *stateless*: l'utente corrente è passato dalla view e risolto dalla sessione;
- la logica risiede nelle entità (GRASP Information Expert).

La tabella 3.1 riassume le classi partecipanti dello scope ristretto a UC-04 e all'autenticazione come prerequisito.

Use Case	Boundary	Control	Entity
UC-04 Ordina un Prodotto	MarketplaceView, PaymentView	OrdinaProdottoController	Ordine, Prodotto, BuonoPromozionale, Utente
Autenticazione (pre-requisito)	LoginView	AutenticazioneController	Utente

Tabella 3.1: VOPC: tracciabilità Use Case → classi partecipanti (Boundary/Control/Entity).

La Figura 3.1 mostra il VOPC di UC-04 secondo il modello *Boundary–Control–Entity* puro: le boundary `MarketplaceView` e `PaymentView` (a cui si collega l'attore Cliente), il control `OrdinaProdottoController` (unico per caso d'uso) e le entity di dominio `Ordine`, `Prodotto`, `BuonoPromozionale` e `Utente`. Le boundary esterne `PaymentGateway` (autorizzazione pagamento, passo 6) e `VenditoreNotifier` (notifica asincrona al Venditore) chiudono i confronti con il mondo esterno. Nel diagramma sono presenti SOLO attributi strutturali rilevanti e operazioni di *business*, senza getter/setter e senza dettagli implementativi (niente reverse engineering). I Bean/DTO e i DAO appartengono al *design-level*: compaiono nel class diagram e nei sequence diagram di progettazione, non in questa vista di analisi.

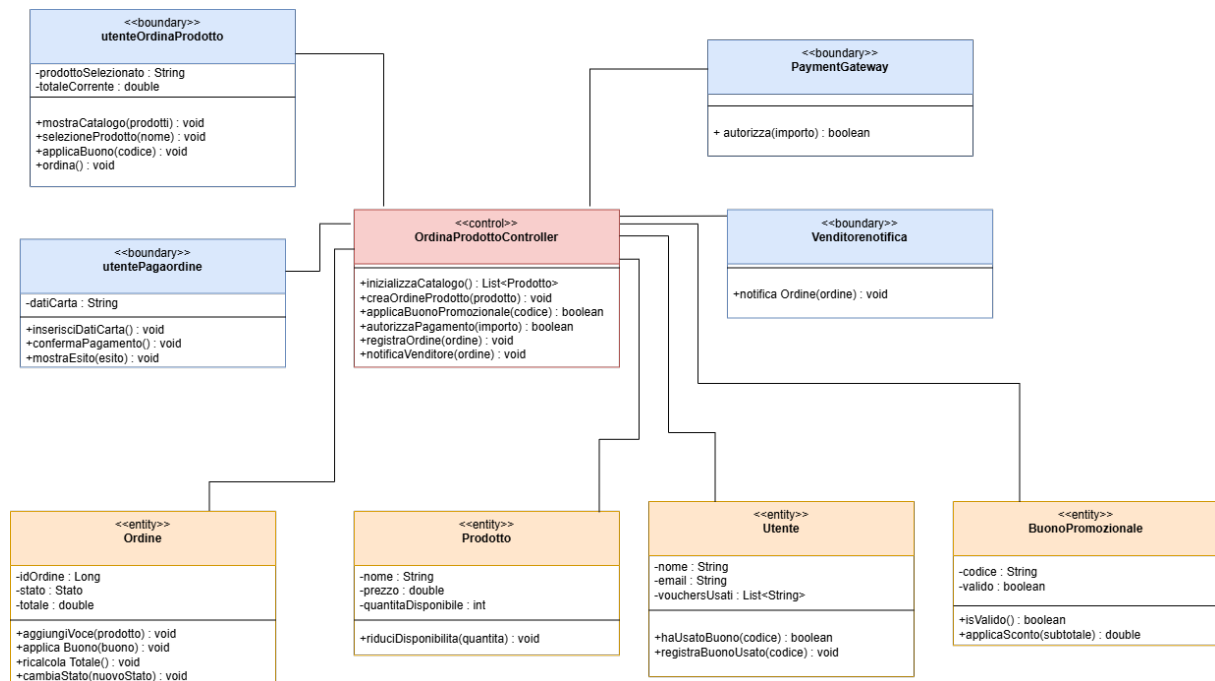


Figura 3.1: VOPC (analisi BCE pura) del caso d'uso Ordina un Prodotto: solo Boundary, Control ed Entity con attributi rilevanti e operazioni di business; attori Cliente/Venditore a margine, Bean/DAO a design-level.

3.2 Design-Level Class Diagram

Il **Design-Level** ingegnerizza la soluzione emersa dal VOPC (analisi): introduce le **interfacce**, i **tipi effettivi Java**, le **eccezioni** e i **pattern GoF** per risolvere requisiti non funzionali come estendibilità, disaccoppiamento dei layer e persistenza dinamica (NFR-01). Per la leggibilità, il diagramma di progettazione è mostrato focalizzato sui sotto-sistemi per pattern, che sono anche i punti d'esame delle domande di De Angelis (disallineamento *analisi vs design*).

3.2.1 Abstract Factory: persistenza (NFR-01)

Intento — fornire un'interfaccia per creare famiglie di oggetti correlati senza specificare le classi concrete. La persistenza è intercambiabile a runtime: **DAOFactory** è l'Abstract Factory, mentre **DemoDAOFactory**, **FSDAOFactory** e **JBCDAOFactory** sono le factory concrete che espongono i DAO coerenti (utente, prodotto, ordine, buono) come *Abstract Products*. I DAO sono istanziati esclusivamente dalla factory, mai con **new** nel controller.

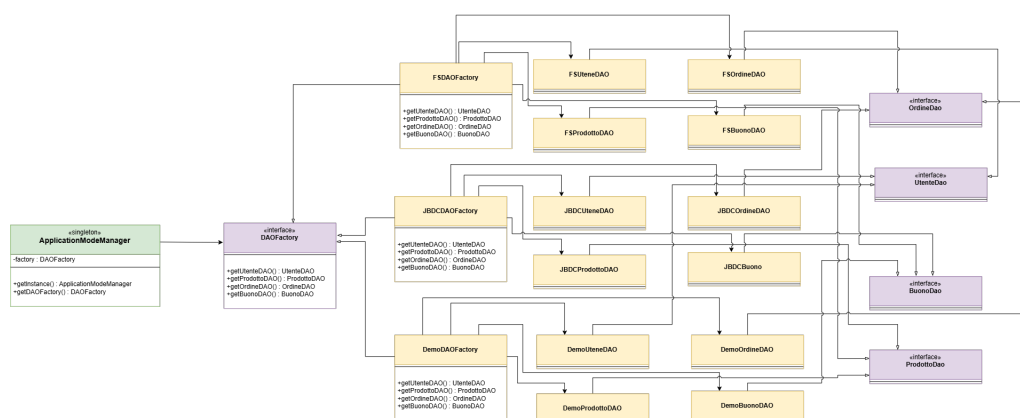


Figura 3.2: Design-level: Abstract Factory della persistenza (DAOFactory + concrete factory Demo/FS/JDBC e interfacce DAO).

3.2.2 Pattern Strategy: scontistica del buono

Intento — definire una famiglia di algoritmi intercambiabili e incapsularli dietro un'interfaccia stabile. Gli sconti del buono promozionale sono calcolati tramite la **ScontoStrategy** (concrete: percentuale, importo fisso), impiegata dal **Context Ordine** in **ricalcolaTotale()** e costruita dalla Simple Factory **ScontoStrategyFactory**.

3.2.4 Pattern Singleton: sessione e modalità

`SessionManager` (sessione utente) e `ApplicationModeManager` (modalità applicativa) garantiscono un'unica istanza accessibile globalmente (*Holder Idiom*, thread-safe). `ScontoStrategyFactory` espone un metodo statico di creazione come semplice punto di accesso.

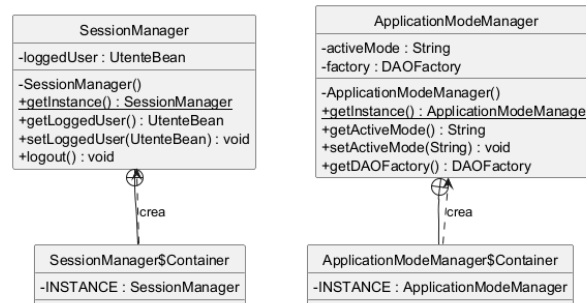


Figura 3.5: Design-level: Singleton della sessione e della modalità applicativa.

3.2.5 Macchina a stati dell'Ordine (BR-04)

La macchina a stati BR-04 è implementata nell'entità `Ordine`, nel metodo `cambiaStato()`. Il metodo ammette solo le transizioni previste e, dopo una transizione valida, notifica gli osservatori. Non viene introdotto un ulteriore pattern GoF: il codice usa un'enumerazione per rappresentare lo stato e una guardia per validare ogni passaggio. La specifica degli stati e dei trigger è descritta nella modellazione comportamentale (Sezione 3.3, Figura 3.8).

3.3 Modellazione Comportamentale

3.3.1 Activity Diagram (Ordina un Prodotto)

Il diagramma di attività descrive il flusso del caso d'uso UC-04: caricamento del catalogo, selezione del prodotto, applicazione opzionale del buono (4a), accesso al pagamento e gestione degli esiti positivi o negativi.

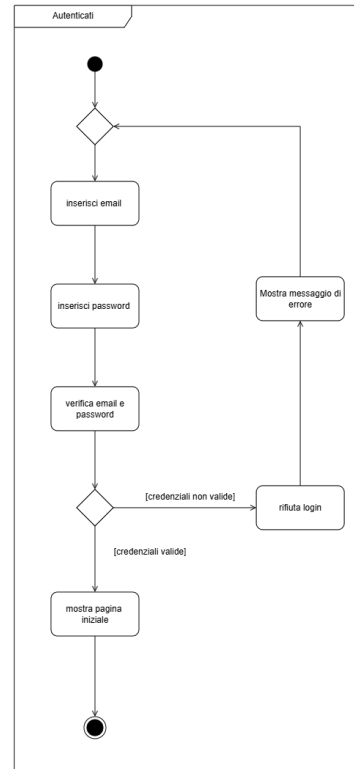
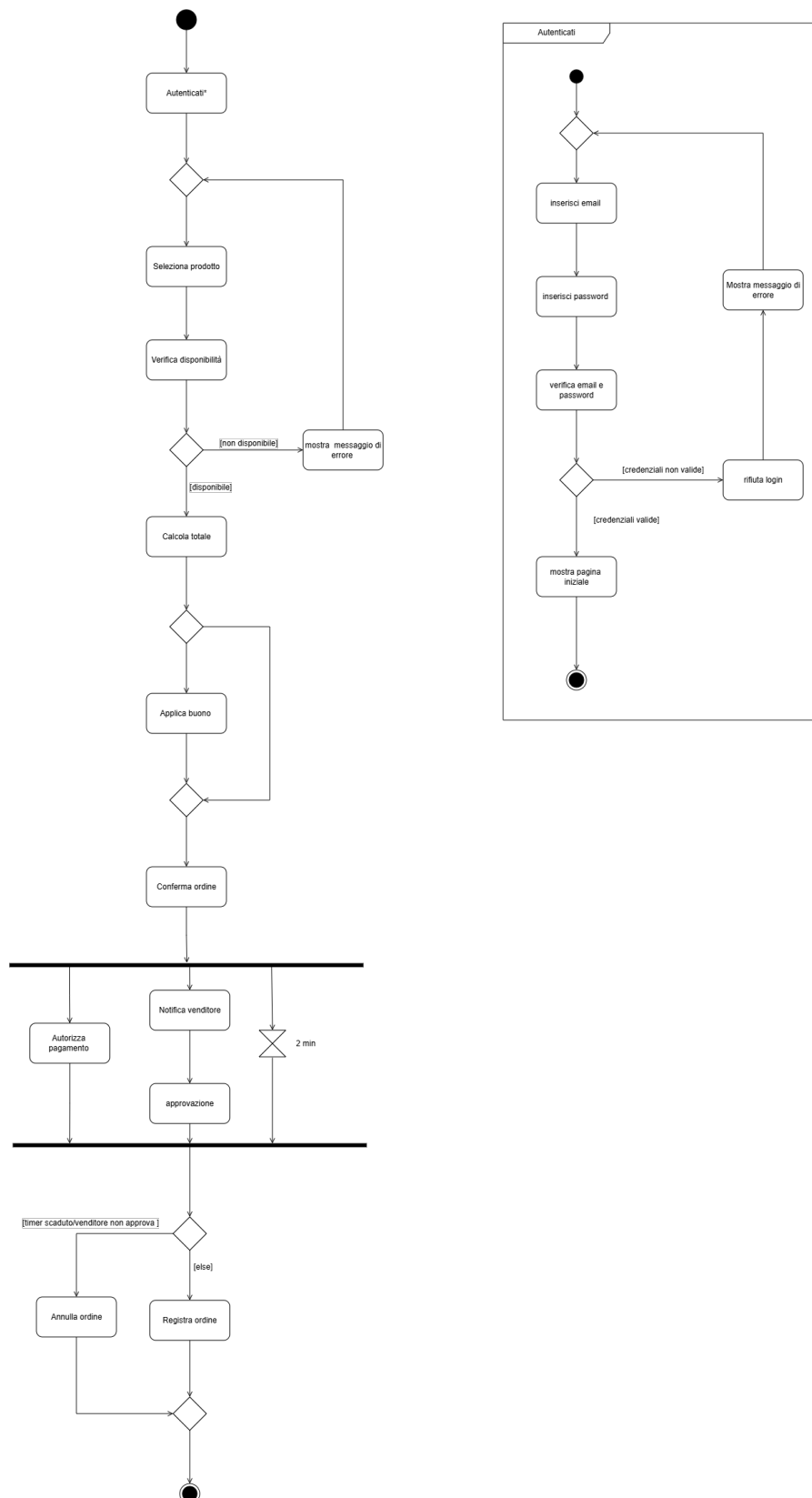


Figura 3.6: Activity diagram del caso d'uso Ordina un Prodotto.

3.3.2 Sequence Diagram (Ordina un Prodotto)

Il sequence diagram segue la **catena a gradino** standard (Lezione 32): l'attore **Cliente** si autentica, la boundary **MarketplaceView** istanzia il control con «create» `Initialize OrdinaProdottoController()`, poi la selezione del prodotto crea l'**OrdineBean**, rispettando il pattern **BCE** con i bean ai confini. Le lifeline sono: **Cliente** → **MarketplaceView**, **PaymentView** (boundary), **OrdinaProdottoController** (control), i bean **OrdineBean**, **UtenteBean**, **PaymentInfoBean**, le boundary esterne **BuonoDAO**, **ProdottoDAO**, **OrdineDAO** e **PaymentGateway**, le entity **BuonoPromozionale**, **Prodotto**, **Ordine** e l'osservatore **VenditoreNotifier** sino all'attore **Venditore**. Le frecce riportano i *nomi reali dei metodi Java*: le sincrone sono piene a punta piena (il chiamante attende), le asincrone con stanghetta aperta e i ritorni tratteggiati.

La **MarketplaceView** e la **PaymentView** non si chiamano direttamente: la seconda è creata dalla prima passando il bean («create» `Initialize PaymentView(ordineBean)`), rispettando l'isolamento BCE. L'estensione *buono promozionale* (4a) è un frammento opt che invoca `applicaBuonoPromozionale(codice, bean, utente)`: valida il codice (`BuonoDAO.findByCodice, isValido`) e ritorna l'**OrdineBean** col totale scontato. **OrdinaProdottoController** ha due attivazioni disgiunte (*deactivate* tra le due invocazioni dell'utente), evitando due sincrone in ingresso nello stesso box: una per `applicaBuonoPromozionale` e una per `processaPagamento`.

Nel flusso principale la **PaymentView** invoca `processaPagamento(bean, utente, paymentInfoBean)` crea il **PaymentInfoBean** e richiede `paymentGateway.autorizza(importo)` (passo 6). Nel ramo alt autorizzato il sistema esegue `prodottoDAO.findByNome, prodotto.riduciDisponibilita(1), prodottoDAO.update, «create» Initialize Ordine(), ordine.aggiungiVoce, ordineDAO.save` e infine `ordine.notificaSottomissione()`: la sincrona si ferma al **VenditoreNotifier** (boundary) che invia la notifica asincrona `onStatoCambiato(ordine)` all'attore **Venditore** senza activation box (best-practice). Se l'autorizzazione è rifiutata (estensione **6a**) il controller solleva **BusinessValidationException** gestita dalla **PaymentView** inline (si resta su **On Payment**), senza creare ordini "fantasma".

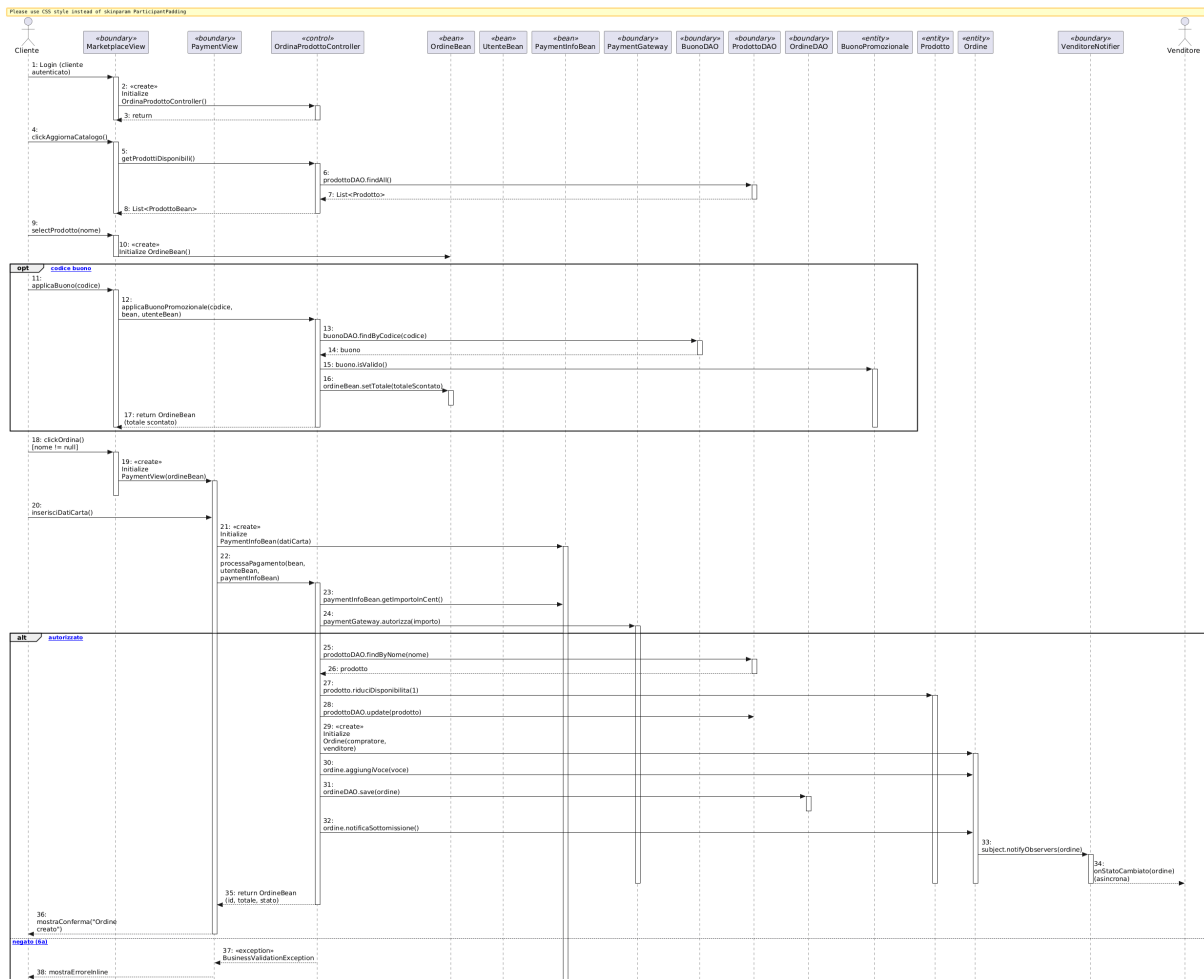


Figura 3.7: Sequence diagram del caso d'uso Ordina un Prodotto: catena a gradino, estensione buono (4a) in opt, creazione di PaymentView, alt autorizzazione (passo 6) con riduciDisponibilita, ordineDAO.save e notifica asincrona onStatoCambiato() tramite Observer; ramo else per l'estensione 6a.

3.3.3 State Diagram (Ordina un Prodotto, navigazione UI)

Lo state diagram segue le regole di stile del corso (Lezione 39): è una macchina a *schermate* UI che si apre sulla *Home Page* e si richiude su di essa, con tre stati piatti *On Home* → *On Marketplace* → *On Payment*. Ogni transizione è scritta nel formato **trigger** [**guardia**] / **behavior** (la prima e l'ultima senza etichetta). Gli scenari coperti derivano dal codice reale: caricamento del catalogo, selezione del prodotto, voucher/buono promozionale (estensione 4a), pagamento (passo 6) e le estensioni di errore (3a scorte, 6a rifiuto). Le guardie sono mutuamente esclusive (determinismo); gli errori restano nella view corrente come *self-loop* con aggiornamento inline del messaggio (design-code consistency).

On Marketplace (catalogo, selezione, buono):

- self-loop *caricamento catalogo*: click [btnAggiorna] / getProdottiDisponibili().

- self-loop *prodotto non selezionato*: `clickOrdina [prodotto nullo] / mostraErrore("Seleziona un prodotto")`.
- self-loop *prodotto rimosso*: `clickOrdina [prodotto non trovato] / mostraErrore("Non più disponibile")`.
- self-loop *estensione 4a* (voucher), guardie disgiunte su `applicaBuono`:
 - `[codice vuoto] / mostraErrore("Codice manca");`
 - `[codice ok && !buonoValido] / mostraErroreBuono();`
 - `[codice ok && buonoValido] / applicaSconto() (resta su On Marketplace)`.
- `→ On Payment: clickOrdina [prodotto presente] / displayPayment()`.
- `→ On Home: click [sidebar Dashboard] / displayHome()`.

On Payment (dati carta, pagamento):

- self-loop *ordine nullo*: `clickPaga [ordine == null] / mostraErrore("Nessun ordine")`.
- self-loop *campi non validi*: `clickPaga [campi non validi] / mostraErroreCampi() (CVV 3 cifre)`.
- self-loop *estensione 3a* (scorte insufficienti): `clickPaga [scorte insufficienti] / mostraErroreScorte()`.
- self-loop *estensione 6a* (rifiutato): `clickPaga [!autorizzato] / mostraErrorePagamento()`.
- `→ On Marketplace: clickAnnulla [ok] / resetCheckout()`.
- `→ On Home: click [sidebar Dashboard] / displayHome()`.
- `→ On Marketplace: clickPaga [ok && autorizzato && scorte ok] / processaPagamento() && saveOrdine() && notificaVenditore()`.

Come nel codice, dopo il pagamento riuscito la vista torna al marketplace (`Navigator.switchTo("marketplace")`). La Home resta l'unico stato di apertura e chiusura, e da **On Payment** le transizioni funzionali (annulla e successo) confluiscono entrambe al Marketplace.

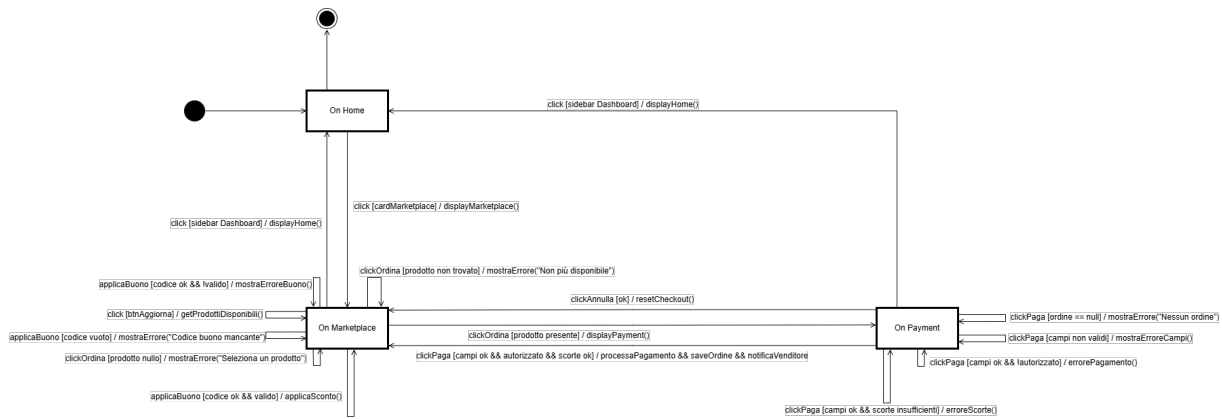


Figura 3.8: State diagram (navigazione UI) del caso d’uso Ordina un Prodotto secondo la Lezione 39: guardie mutuamente esclusive su **On Marketplace** e **On Payment**; successo e annullamento confluiscono al **Marketplace**, la Home resta lo stato di apertura e chiusura.

3.4 Realizzazione del Buono Promozionale

Il buono promozionale (estensione di UC-04) è implementato con il pattern **Strategy**: l'interfaccia `ScontoStrategy` incapsula l'algoritmo; `ScontoPercentualeStrategy` and `ScontoImportoFissoStrategy` sono le strategie concrete; `ScontoStrategyFactory` ricostruisce la strategia dai dati piatti della persistenza. `Ordine` fa da *Context* (`applicaBuono()` + `ricalcolaTotale()`). Il controller `OrdinaProdottoController` coordina le validazioni (esistenza, scadenza, appartenenza, monouso). La persistenza del buono è gestita da `BuonoDAO` nelle tre modalità (Demo, FS JSON, JDBC).

Capitolo 4

Testing

La verifica delle funzionalità e dei requisiti del caso d'uso **UC-04 Ordina un Prodotto** (con autenticazione come prerequisito) è effettuata tramite una suite di test automatizzati **JUnit 5** (*Jupiter*) eseguita da **Maven** (`mvn verify`) con la copertura misurata da **JaCoCo**. La build è riproducibile e la soglia di copertura è imposta come *quality gate*.

4.1 Strumenti e organizzazione

I test sono scritti con **JUnit 5** e risiedono in `src/test/java`, percorso mirror dell'architettura **BCE** del codice applicativo (`src/main/java`). La suite verifica i layer *control* (`OrdinaProdottoController`, `AutenticazioneController`), *entity* (`Ordine`, `Prodotto`, `BuonoPromozionale`), i **pattern** di dominio (`Strategy` degli sconti, `Observer`, `Singleton`, `factory`) e la **persistenza** (**DAO Demo** e **FS**). I DAO **JDBC** (richiedono un MySQL reale) sono esclusi dalla copertura automatizzata e validati manualmente, come documentato nel Capitolo 6.

Il plugin `jacoco-maven-plugin` misura la copertura e impone il *quality gate*: la build fallisce se la copertura del contatore **LINE** scende sotto l'70%. Gli stessi target esclusi (boundary JavaFX che richiedono display, bootstrap e DAO JDBC) sono dichiarati nella configurazione del plugin.

4.2 Specifiche dei casi di test

Di seguito sono descritti i casi di test della suite, che coprono il flusso completo dell'UC-04.

4.2.1 Bean e DTO

Il test **BeanTest** verifica la corretta creazione e il ciclo di vita dei **Bean** che attraversano i confini BCE (`OrdineBean`, `UtenteBean`, `ProdottoBean`, `PaymentInfoBean`): costruttori, valorizzazione iniziale e getter/setter.

4.2.2 Controller applicativi

La suite `OrdinaProdottoControllerTest` (il test del caso d'uso core) verifica in stile *We tested that*:

- *submit ordine con utente nullo*: rifiutato quando non esiste una sessione autenticata;
- *submit ordine prodotto non trovato*: un ordine su un prodotto inesistente nel catalogo viene scartato;
- *submit ordine venditore dal prodotto*: il venditore è risalito dal prodotto acquistato (whole-part) e associato correttamente all'ordine;
- *riduzione disponibilità*: dopo l'acquisto la scorta del prodotto è decrementata di 1 (RF-3);
- *quantità eccedente la scorta*: un acquisto oltre le quantità disponibili solleva un'eccezione (estensione 3a);
- *pagamento autorizzato*: un pagamento approvato dal `PaymentGateway` porta alla creazione dell'ordine in stato `CREATED`;
- *pagamento rifiutato*: un rifiuto del gateway solleva `BusinessValidationException` senza creare ordini (estensione 6a).

Inoltre:

`AutenticazioneControllerTest` verifica autenticazione e registrazione (credenziali valide/invalidhe, creazione profilo con il ruolo, sessione);

`OrdineControllerTest` verifica il ciclo di vita dell'ordine, la transizione di stato (BR-04) e la notifica Observer.

4.2.3 Entità e dominio

- `OrdineTest`: transizione di `cambiaStato`, applicazione del buono, ricalcolo totale e notifica;
- `ProdottoTest`: riduzione della disponibilità e regole di quantità;

4.2.4 Pattern e factory

- `BuonoPromozionaleStrategyTest` e `ScontoStrategyFactoryTest`: scontistica percentuale/importo fisso e Simple Factory (incluso il ramo di errore su tipo ignoto);
- `OrdineLazyFactoryTest`, `PatternTest`: factory lazy e pattern Observer/strategy;

- **ControllerNoArgConstructionTest**: tutti i controller sono istanziabili col costruttore no-arg via ServiceLocator (BCE).

4.2.5 Persistenza

- **DemoDAOTest** e **FSDaoTest**: verifica dei DAO Demo (in-memory) e Filesystem (JSON);
- **ApplicationModeManagerTest**: risoluzione della modalità applicativa (Demo/FS/JDBC) e idempotenza del seed.

4.3 Tracciabilità Requisiti-Test

La matrice collega i Requisiti Funzionali dell'UC-04 (Capitolo 1) alle classi di test JUnit che li verificano.

Requirement	Passo UC-04	Classe di test
RF-3 / US-3	selezione e ordine	OrdinaProdottoControllerTest
	passo 3a (scorte)	OrdinaProdottoControllerTest
	passo 6 (pagamento)	OrdinaProdottoControllerTest
	6a (pagamento negato)	OrdinaProdottoControllerTest
	4a (buono)	BuonoPromozionaleStrategyTest, ScontoStrategyFactoryTest
	BR-04 (stato ordine)	OrdineControllerTest
	Autenticazione	AutenticazioneControllerTest
	Persistenza	DemoDAOTest, FSDaoTest

Tabella 4.1: Matrice di tracciabilità Requisiti-Test (JUnit 5/JaCoCo).

4.4 Risultati dell'esecuzione

La suite viene eseguita con `mvn verify`. L'ultima esecuzione (report JaCoCo, post scope-restrict UC-04) riporta:

Metrica	Valore
Test eseguiti	69
Passati	69
Falliti	0
Errori	0
Copertura INSTRUCTION	73.6%
Copertura LINE	74.4%
Copertura BRANCH	57.7%
Copertura METHOD	74.6%
Copertura CLASS	96.2%

Tabella 4.2: Risultato complessivo della suite di test (scope UC-04).

Tutti i **69 test** passano con **0 errori** e **0 fallimenti**. La copertura **LINE 74.4%** supera la soglia richiesta (70%, imposta nel plugin: la build fallisce in caso contrario). La copertura **CLASS 96.2%** conferma che quasi tutte le classi applicative dello scope UC-04 sono esercitate.

4.5 Copertura per sotto-sistema

La copertura è concentrata sul caso d'uso core: controller `OrdinaProdottoController` e `AutenticazioneController`, entity `Ordine`, `Prodotto`, `BuonoPromozionale`, pattern (Strategy/Observer) e persistenza Demo/FS superano la soglia, a garanzia dei requisiti del UC-04.

4.6 SonarCloud e quality gate

Come nelle relazioni di riferimento, è utilizzata l'integrazione **SonarCloud** per l'analisi statica (bugs, vulnerabilities, code smells e duplicati). Il progetto supera il *Quality Gate* configurato ed è analizzabile con `mvn sonar:sonar` (plugin `sonar-maven-plugin` già presente nel `pom.xml`).

Nota di consegna: da includere lo screenshot del *Quality Gate* di SonarCloud (stato PASSED) e l'URL pubblico del progetto.

Capitolo 5

Exceptions

Il progetto introduce un piccolo insieme di eccezioni applicative per separare gli errori di *validazione del dominio* (regole di business) dalla gestione tecnica dell'infrastruttura. La gerarchia è organizzata attorno alla classe base `BusinessValidationException`; ogni eccezione è *checked* e viene catturata a livello di `Boundary` per la presentazione del messaggio all'utente, senza mai propagare stack infratilistici fuori dai confini BCE.

5.1 `BusinessValidationException`

`BusinessValidationException` estende `java.lang.Exception` ed è la radice delle violazioni di regola di business nel progetto. Viene sollevata ogni volta che l'input dell'utente (o lo stato del dominio) viola una regola applicativa, ad esempio:

- `OrdinaProdottoController` la solleva quando l'autorizzazione del pagamento è rifiutata (estensione **6a**) o le scorte sono insufficienti (estensione **3a**);
- la validazione dei campi carta (`validaCampi`) la usa per segnalare campi mancanti o CVV non valido;
- il buono promozionale non valido (inesistente, scaduto, già usato, venditore errato) ritorna `BusinessValidationException`.

Gradualmente le `Boundary` la catturano e mostrano il messaggio in una `Label` inline (nessuna view fittizia), mantenendo l'utente nella schermata corrente.

5.2 `AutenticazioneException`

Sottoclasse di `BusinessValidationException` che racchiude gli errori di **autenticazione** e **registrazione**: credenziali errate, utente o password non valide, violazioni sulla creazione del profilo. È usata dal controllare `AutenticazioneController` per uniformare i messaggi di login.

5.3 InvalidStateTransitionException

Sottoclasse di `BusinessValidationException` usata dalla macchina a stati dell'`Ordine` (BR-04): quando si tenta una transizione di stato non prevista (es. passare direttamente da `CREATED` a `DELIVERED`), `Ordine.cambiaStato()` solleva `InvalidStateTransitionException`, bloccando l'operazione illegale e conservando l'integrità del modello.

5.4 Gestione nelle Boundary

Tutte le eccezioni applicative sono catturate nei `handler` delle view JavaFX (es. `catcher BusinessValidationException` nel `setOnAction` della `PaymentView`): il messaggio viene mostrato inline nella *Label* di stato ed è fornito dal costruttore dell'eccezione; ciò garantisce coerenza con la modellazione BCE (le Entity non dipendono da GUI) e con lo State Diagram a *schermate*.

Nota: gli errori di infrastruttura invece (es. `SQLException`) sono gestiti dai DAO e non si propagano alle view, come documentato nel Capitolo 6; il Quality Gate impone test automatizzati per tutti i rami di errore (Capitolo 4).

Capitolo 6

Persistenza

La persistenza di **CiboAmico** è progettata per essere *intercambiabile a runtime* (NFR-01): la stessa logica applicativa funziona con tre backend di memorizzazione senza alcuna modifica ai controller, grazie al pattern **Abstract Factory**. Il requisito è realizzato dal componente `Persistence` in `src/main/java/.../persistence`.

6.1 Pattern Abstract Factory

L'interfaccia `DAOFactory` definisce i metodi di creazione dei DAO (`UtenteDAO`, `ProdottoDAO`, `OrdineDAO`, `BuonoDAO`). Le tre concrete factory `DemoDAOFactory`, `FSDAOFactory` e `JDBCDAOFactory` restituiscono implementazioni coerenti dello stesso set di DAO.

Il `ApplicationModeManager` (**Singleton**) seleziona la factory corretta a runtime in base alla modalità attiva:

```
factory = switch (activeMode) {  
    case MODE_JDBC -> new JDBCDAOFactory();  
    case MODE_FS   -> new FSDAOFactory();  
    case MODE_DEMO -> new DemoDAOFactory();  
}
```

Tutti i controller accedono ai DAO esclusivamente tramite `getDAOFactory()`: la decisione di *quale* backend usare è confinata in `ApplicationModeManager`, mantenendo i controller indipendenti dalla tecnologia di persistenza (coesione e disaccoppiamento BCE).

6.2 Modalità Demo (in-memory)

`DemoDAOFactory` produce DAO che tengono i dati in un *mappa in memoria* (seed demo idempotente al primo avvio). È la modalità predefinita, utile per la demo e per i test automatizzati: nessun file né database esterno, riavvio sempre coerente con i dati demo (`DemoDAOTest`).

6.3 Modalità Filesystem (JSON)

FSDA0Factory produce DAO che serializzano le entità su **file JSON** locali tramite Gson (configurato in GsonConfig). Ogni DAO (FSUtenteDAO, FSProdottoDAO, FSOrdineDAO, FSBuonoDAO) gestisce il caricamento/salvataggio delle rilevanti collezioni. La modalità è coperta da FSDaoTest.

6.4 Modalità JDBC (MySQL)

JDBCDAOFactory produce DAO che interagiscono con un **database MySQL** tramite JDBC. Lo schema relazionale è definito in `sql/CiboAmico.sql` e comprende, per UC-04, le tabelle: *utenti*, *ruoli*, *prodotti*, *ordini* e *voci_ordine*.

- la connessione è centralizzata in `ConnectionManager`, che legge le credenziali da *system properties* (`ciboamico.db.url`, `ciboamico.db.user`, `ciboamico.db.password`) con fallback a valori di sviluppo locali;
- le transazioni (es. ordine + riduzione scorte) sono gestite atomicamente con `setAutoCommit(false)` e `commit/rollback`;
- i DAO JDBC **non partecipano alla copertura automatizzata** (esclusi dal JaCoCo perché richiedono un MySQL reale) e sono validati manualmente, come documentato nel Capitolo 4.

6.5 Coerenza con il sequence e gli use case

La scelta del DAO nel controller è visibile nel *sequence diagram* (Capitolo 3): `OrdinaProdottoController` interroga `OrdineDAO`, `ProdottoDAO`, `BuonoDAO` e `UtenteDAO` attraverso l'Abstract Factory, per poi delegare la persistenza dell'ordine a `ordineDAO.save`. La tracciabilità persistenza-test è coperta da `DemoDAOTest` e `FSDaoTest` (Capitolo 4).

Capitolo 7

Codice e Qualità

Questo capitolo descrive l'organizzazione del codice e le attività di qualità applicate al progetto **CiboAmico**. La verifica della correttezza è affidata a una suite di **test JUnit 5** con **copertura JaCoCo** (quality gate $LINE \geq 80\%$), come richiesto dai criteri di Falessi e De Angelis; la qualità è inoltre garantita dall'adesione ai **design pattern GoF** e all'architettura **Boundary–Control–Entity**, che riducono il **technical debt** strutturale.

7.1 Struttura del codice

Il codice sorgente (`src/main/java`) è organizzato per package BCE e per pattern, ciascuno con una responsabilità netta e verificabile:

Package	Ruolo BCE	Contenuto
boundary	Boundary	View JavaFX (<code>MarketplaceView</code> , <code>LoginView...</code>), <code>ViewFactory</code>
boundary/cli	Boundary CLI	Abstract Factory della famiglia CLI (<code>CLIViewFactory</code> , <code>CLIContext</code>)
control	Control	Controller applicativi: <code>OrdinaProdottoController</code> , <code>TrovaRicetteController...</code>
bean	Bean/DTO	<code>OrdineBean</code> , <code>ProdottoBean</code> , <code>UtenteBean...</code>
entity	Entity	<code>Ordine</code> , <code>Prodotto</code> , <code>BuonoPromozionale...</code>
persistence	Persistenza	Interfacce DAO e implementazioni <code>demo/fs/jdbc</code>
pattern	GoF	<code>observer</code> , <code>strategy</code> , <code>factory</code> , <code>singleton</code> , <code>adapter</code> , <code>facade</code>
exception	-	Eccezioni di dominio (<code>BusinessValidationException</code> , ...)

Tabella 7.1: Organizzazione a package di CiboAmico.

L'architettura mantiene le Boundary **senza accesso diretto alla persistenza**: i Controller applicativi espongono un costruttore no-arg che risolve la DAOFactory dal **ServiceLocator** `ApplicationModeManager.getInstance()` (in modalità DEMO di default), e le View creano il proprio Controller con `new Controller()` senza conoscere la persistenza — in linea con i progetti 30/30 di riferimento. L'unica eccezione è `boundary.cli`, che realizza l'**Abstract Factory** della famiglia CLI.

7.2 Metodologia di sviluppo

Il codice è stato sviluppato secondo i principi del **design pattern GoF** e del **GRASP**, con l'obiettivo esplicito di mantenere basso il technical debt:

- ogni **caso d'uso** ha esattamente un controller applicativo stateless (**Creator**, **Controller**, **Information Expert**);
- le **Entity** concentrano la logica di dominio e la validazione (invarianti garantite nei costruttori), secondo l'approccio *Domain-Driven Design*;
- la **persistenza** è inserita tramite **Abstract Factory** (DAOFactory e famiglie Demo/FS/JDBC), così il cambio di persistenza non tocca la logica di business;
- i **pattern creazionali/comportamentali/strutturali** sono introdotti solo dove riducono davvero la complessità (Strategy per gli sconti, Observer per le notifiche).

7.3 Verifica dei design pattern GoF

La conformità ai pattern GoF dichiarati nel Capitolo 3 è verificabile **dal codice**: ogni pattern documentato corrisponde a classi reali presenti nel repository.

Pattern GoF	Classi di riferimento	Categoria
Abstract Factory	DAOFactory + Demo/FS/JBCDAOFactory	Creazionale
DAO	5 interfacce DAO + impl demo/fs/jdbc	Persistenza
Strategy (sconti)	ScontoStrategy + Percentuale/ImportoFisso	Comportamentale
Observer	OrdineSubject + Utente/VenditoreNotifier	Comportamentale
Singleton	SessionManager, ApplicationModeManager	Creazionale
Simple Factory	ScontoStrategyFactory	Creazionale

Tabella 7.2: Pattern GoF impiegati e verifica nel codice.

7.4 Verifica e quality gate

La correttezza è garantita da una suite di test automatizzati ed è riepubblicata convenientemente in fase di build:

```
mvn test      # esecuzione dei test JUnit 5
mvn verify    # test + quality gate di copertura JaCoCo (LINE >= 80%)
```

La soglia **LINE** $\geq 80\%$ è imposta nel plugin `jacoco-maven-plugin` (`verify`): la build fallisce se non si raggiunge, garantendo che refactoring e nuove funzionalità non degradino la copertura. Il dettaglio dei test e la copertura corrente sono riportati nel Capitolo 4.

7.5 Correzioni architetturali applicate

Nel corso dello sviluppo sono state applicate logiche di robustezza che migliorano la qualità senza alterare la responsabilità BCE:

- **Credenziali database non più hardcoded.** La classe `ConnectionManager` centralizza la connessione JDBC leggendo la configurazione da *system properties* (`ciboamico.db.url`, `ciboamico.db.user`, `ciboamico.db.password`) con fallback a valori di sviluppo locali, disaccoppiando le credenziali dal codice.
- **Refactoring del Singleton lazily-configurato.** `OrdineLazyFactory` separa la configurazione (`configure(DAOFactory)`, una sola volta al bootstrap) dall'accesso (`getInstance()`), migliorando la gestione del ciclo di vita del Singleton.
- **Bean/DTO puri.** I Bean sono stati ripuliti dai costruttori no-arg ridondanti: quando il compilatore può generare il costruttore di default, questo non viene dichiarato esplicitamente (i Bean restano deserializzabili da Gson/DAO senza accoppiamento).

Capitolo 8

Video e Link

Il progetto è disponibile pubblicamente e la qualità è monitorata con analisi statica. I seguenti riferimenti completano la documentazione.

8.1 Repository GitHub

Il codice sorgente completo (applicazione JavaFX/CLI, test JUnit 5, script di build e documentazione) è disponibile sul repository pubblico:

<https://github.com/Damiano-o/ciboamico.git>

8.2 SonarCloud

Il progetto è analizzato con **SonarCloud** (organizzazione `ciboamico-ispw`); il pannello pubblico riporta il *Quality Gate*, le metriche di duplicazione, i code smells e la copertura importata da JaCoCo. Il link di riferimento sarà:

<https://sonarcloud.io/organizations/ciboamico-ispw>

La configurazione degli *excludes* dei componenti grafici è dichiarata nel `pom.xml` (sezione `sonar`) per non falsare le metriche di copertura.

8.3 Video dimostrativo

Nota di consegna: il video dimostrativo (`video-ubergata.mp4`) è disponibile nel repository o su piattaforma di condivisione; il link sarà inserito a cura dello studente prima della consegna.